pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Classifying 8-K Items with TF-IDF

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: 8-K sections, labelled by their filers

The eight items in the file

1.01	Material definitive agreement
1.03	Bankruptcy or receivership
2.02	Results of operations
2.03	Direct financial obligation
2.04	Triggering events that accelerate
2.06	Material impairments
3.01	Notice of delisting
5.02	Departure or election of officers

Today

1. Codes as text; the file is a sample
2. Split by filing date
3. The heading leak, shown first
4. The classifier on the bodies
5. Rare items, reported on their own
6. Excel: COUNTIFS on text codes
7. AI check: a random split

5,126 sections from 4,965 filings, each labelled by the item its filer chose.

The item the model assigns

An 8-K model's output

Allowed names

the item the model assigns

the model's probability or score in this file

the filer's item

Never called

a default signal

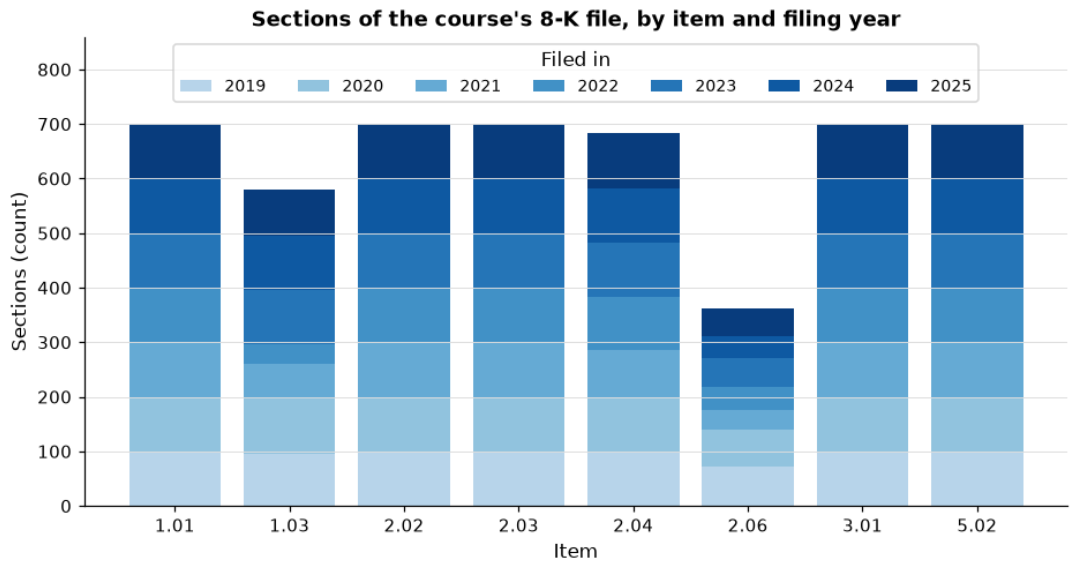
a distress flag or a warning

a credit event or credit view

Every excerpt has its source line. Company names appear only there, as authors. Item 5.02 sections are counted by item; no name from them is printed.

Item codes are text; the file is a sample

```
pd.read_csv(...)["item"]                    # float64 by default: 1.01 as a number
float("2.10")                               # 2.1: a code would lose its last digit
pd.read_csv(..., dtype={"item": str})       # the course reads codes as text
```



Sections in the file

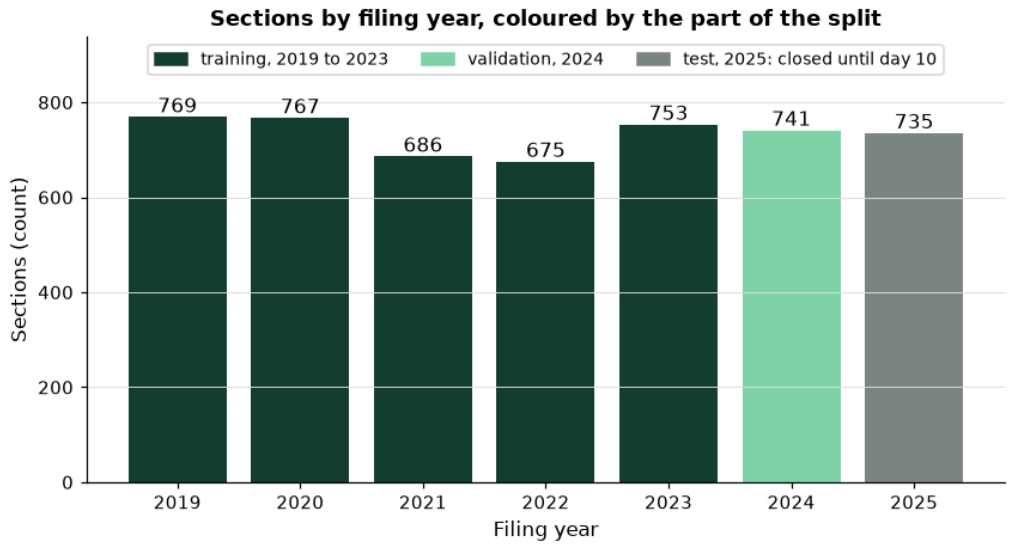
Item 1.03, bankruptcy	**580**
Item 2.04, triggering events	**683**
Item 2.06, the smallest	**363**
each of the five full items	**700**

At most 100 sections per item per year: how rare an item is here is the course's choice. Each text is the body without its heading, cut at 300 words (median 161).

How the split by date works

The problem	A filer repeats its wording quarter after quarter, so a random split puts the same wording on both sides
The cut	Fit on everything filed up to one date, validate on the next year, keep the last year closed
You control	The two cut dates
What comes out	Three tables with no section on two sides
The order	Every validation section was filed after every training section, as new filings arrive

The split by date



Training: 3,650 sections

filed 2019 to 2023, fitted on

Validation: 741 sections

filed in 2024, scored on; 2.06 has 41

Test: 735 sections

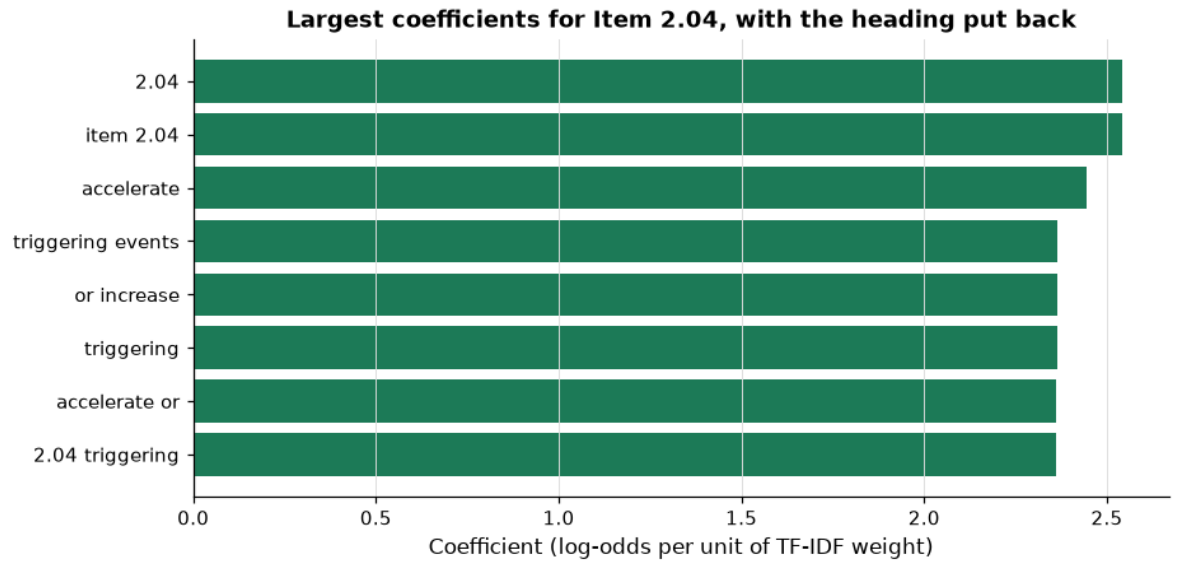
filed in 2025, closed until day 10

```
train, valid, test = textkit.date_split(sections, "filed", "2023-12-31", "2024-12-31")
del test                      # closed until day 10: no later cell can score it
```

A company files the same wording quarter after quarter: a random split puts it on both sides.

The heading leak, shown first

```
Item 2.04 Triggering Events That Accelerate or Increase a Direct Financial Obligation ...
```



0.9899

validation macro F1, heading put back

0.9269

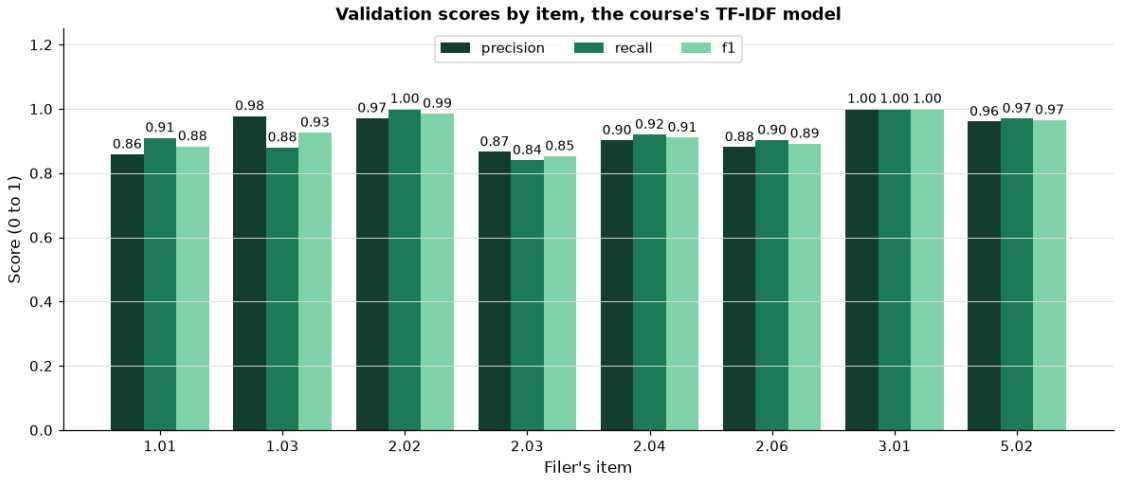
validation macro F1 on the stored bodies

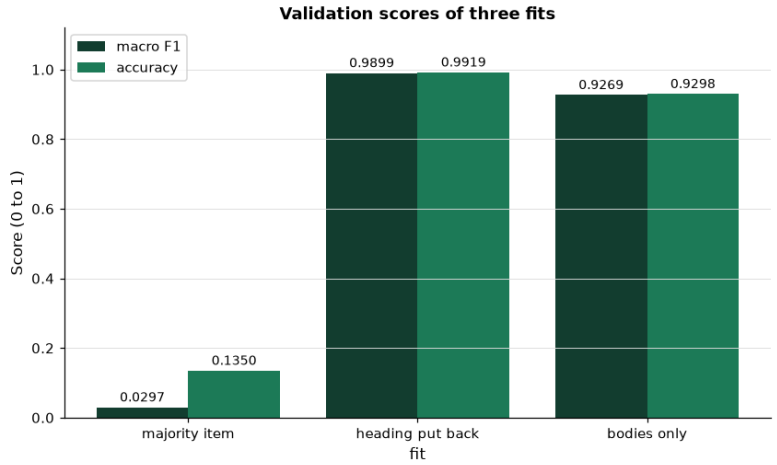
The heading is the label, written in the text. The fix: fit on the bodies the tool stored.

How the classifier works

Two stages	One Pipeline: TF-IDF turns text into weights, a logistic regression learns from them
The scores	Per item, the sum of the section's weights times that item's coefficients
Softmax	Turns the eight scores into eight probabilities that add to 1
The fit	Moves the coefficients until training items get high probabilities, with a penalty holding them back
What comes out	predict gives the item with the largest probability; predict_proba gives all eight

The classifier on the bodies

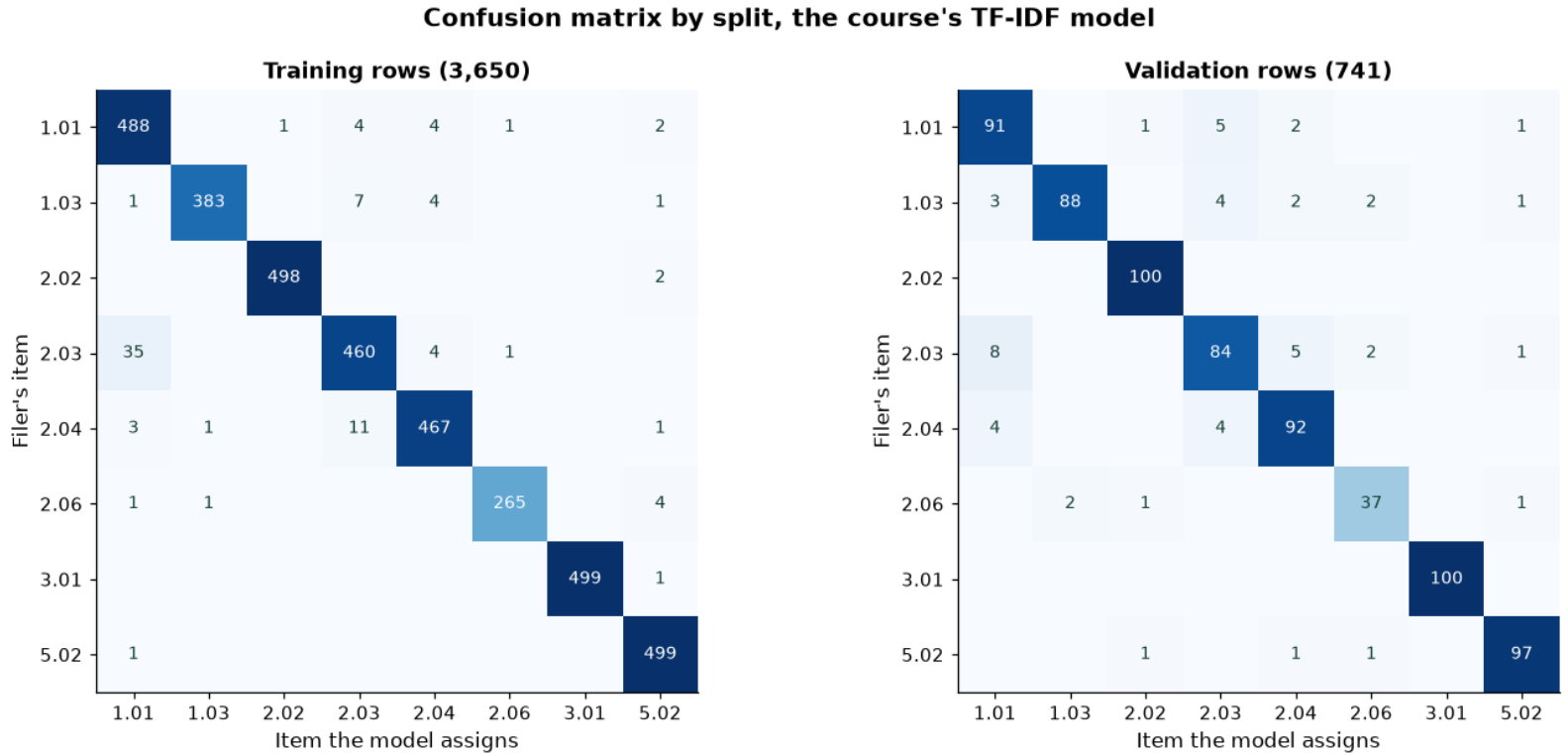




Every section assigned Item 1.01: macro F1 0.0297, accuracy 0.1350. The model on the bodies: macro F1 0.9269, accuracy 0.9298. The tallest pair, with the heading put back, is the leak. TF-IDF inside the Pipeline: 40,403 terms, learned from the training rows only.

Macro F1 gives a rare item the weight of a common one. Accuracy appears only beside the baseline.

Where the misses go



The largest miss

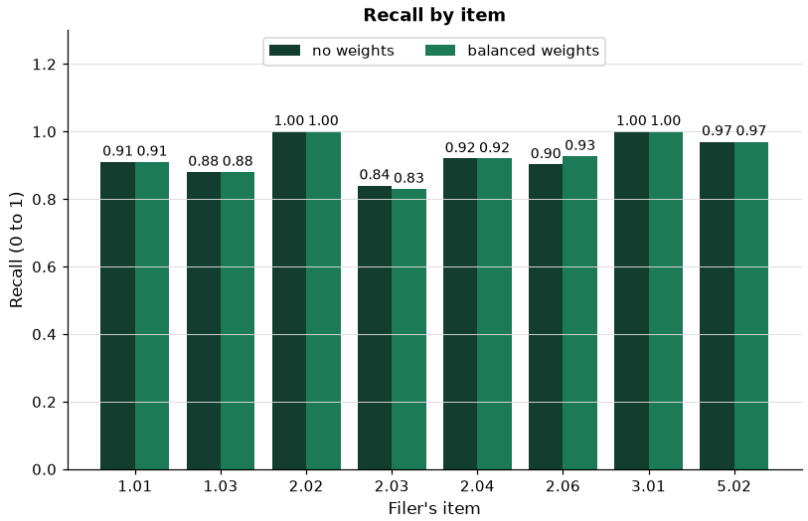
8 validation sections filed as 2.03 are assigned 1.01: a new obligation is often created by an agreement, and the two items share their words.

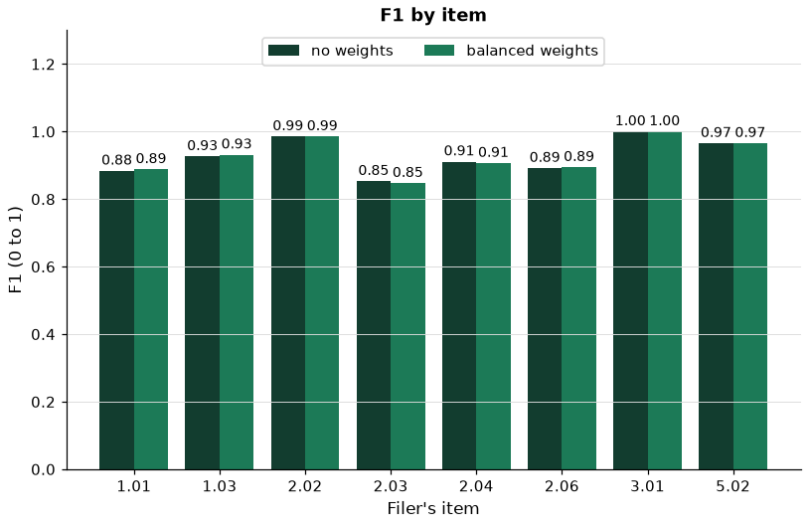
Rows: the filer's item. Columns: the item the model assigns. Test rows closed until day 10.

How precision, recall, and F1 work

The problem	Accuracy lets the common items hide the rare ones
Precision	Of the sections assigned an item, the share the filer filed under it
Recall	Of the filer's sections of an item, the share the model finds
F1 and macro	F1 is the harmonic mean of the two; macro F1 is the plain mean of the eight
What comes out	A table with a row per item and a macro row; class weights can raise a rare item's recall

The rare items, and class weights





Recall

Item 1.03	**0.8800**
Item 2.04	**0.9200**
Item 2.06 (41)	**0.9024**
2.06, balanced	**0.9268**

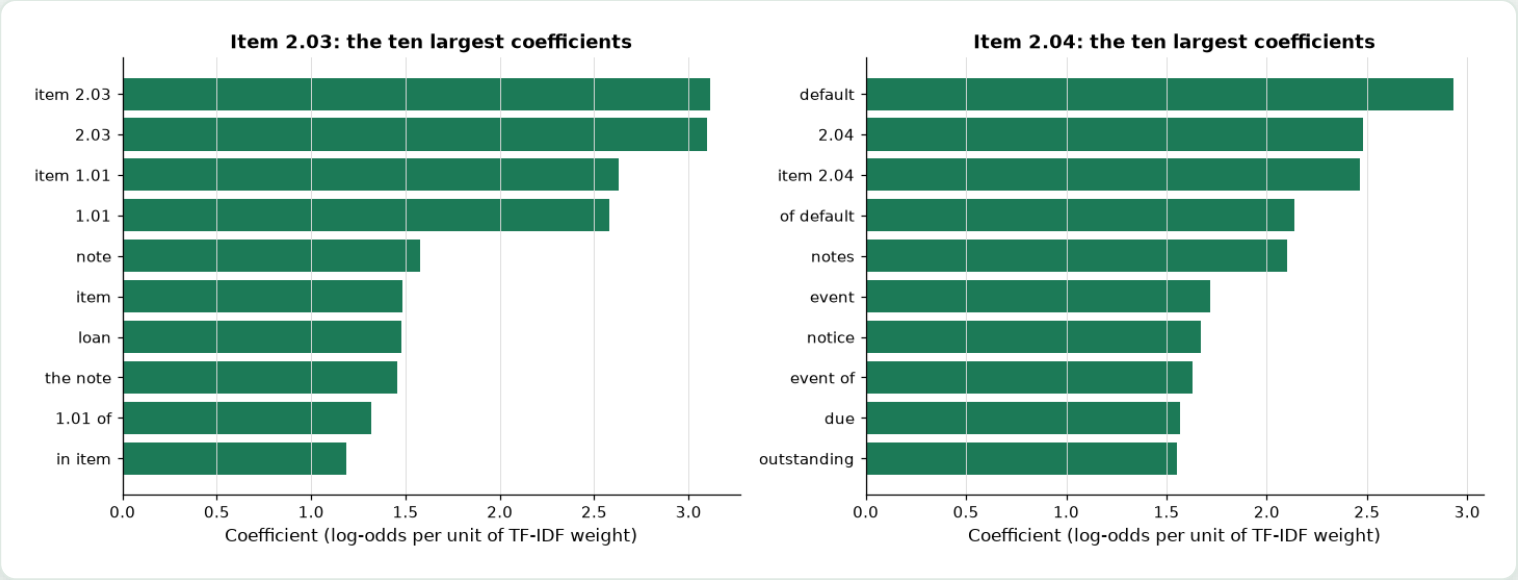
```
textkit.text_classifier(class_weight="balanced")     # each item weighted by 1 / its share
```

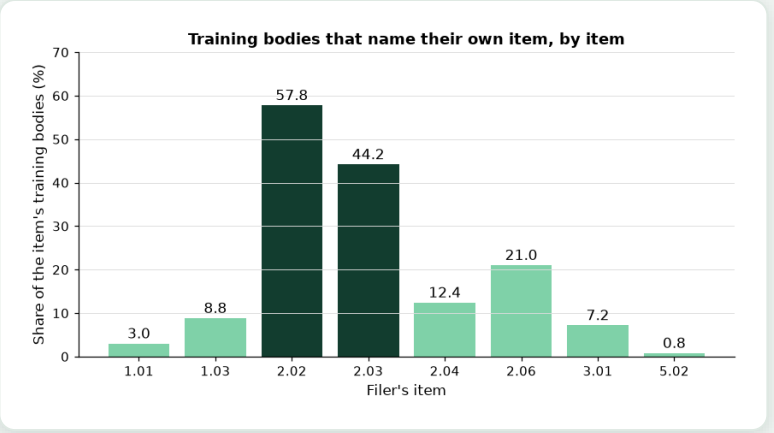
The two models assign a different item to 3 of 741 sections. Chosen on validation by macro F1: tfidf_logistic_balanced, 0.9271 against 0.9269. With weights the outputs are scores, not probabilities. One setting changed: class_weight, None to balanced.

How to read the coefficients

The problem	A fitted model should show what it leans on
A coefficient	Ties one term to one item: large and positive pushes a section toward the item
The push	The term's TF-IDF weight in the section scales it
top_terms	Reads the largest coefficients of one item, with the vocabulary's terms as names
What comes out	A description of what this model leans on in this file, not of what a filing means

What the model leans on





Item 2.02: 0.5780. Item 2.03: 0.4420. Cross-references a filer writes, not the heading.

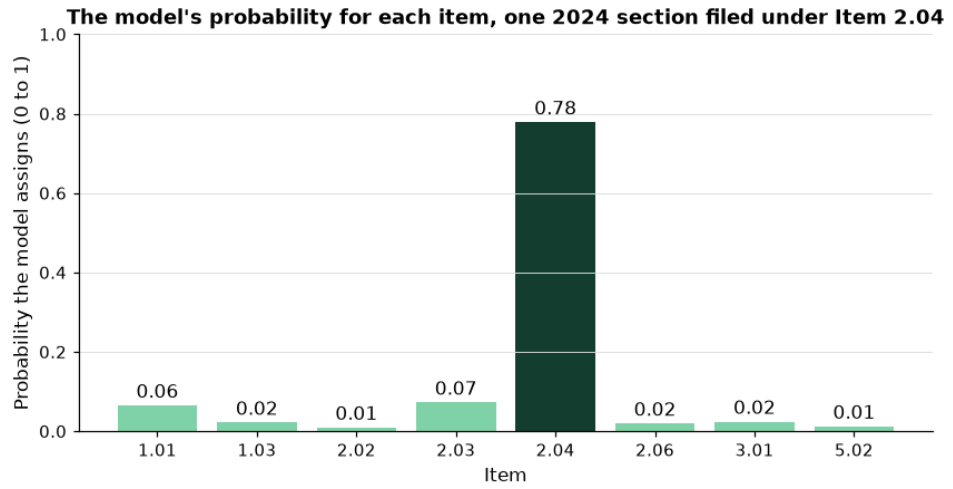
Item numbers removed: macro F1 0.9234 against 0.9269. A coefficient is what the model leans on, not a reason.

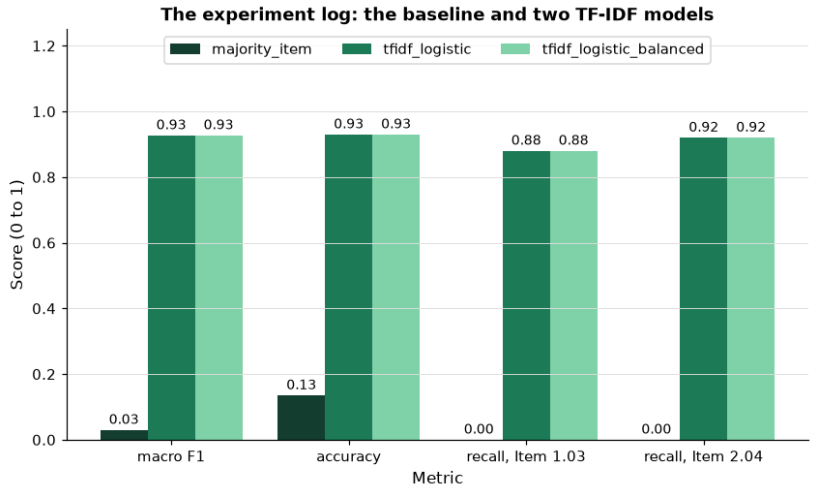
The model output label, and the log

0.7782

probability for Item 2.04, this file

Which of eight headings the text resembles. Never a default signal, a distress flag, or a credit event; nothing about the company.





results.csv: 4 decimals, LF line endings; the heading model is not a candidate.

How the classifier's settings work

Three jobs	What the model sees, how hard it is held back, and how the fit runs
What it sees	The TF-IDF stage: ngram_range, min_df, sublinear_tf
Held back	C, the inverse of the penalty: small is simple, large follows the training rows
How to look	Change one setting alone; score the fitting rows and the validation rows side by side
What comes out	A picture of where a setting starts to overfit: fitting rows rise, validation rows stop

Every setting of the logistic regression

C (default 1.0; tune first)

The inverse of the penalty's strength. Smaller C holds the coefficients back harder.

Smaller: a simpler model that fits the training sections less closely. Larger: coefficients grow and the fit follows the training sections (the figure).

class_weight (default None; a choice)

Weights each item in the loss. None: every section counts the same.

'balanced' weights an item by the inverse of its share of the training rows: the rare items count more (section 3.5).

max_iter (default 100; set once)

The most steps the solver takes. 100.

The course sets 2000 so the fit converges on thousands of columns; too few gives a convergence warning.

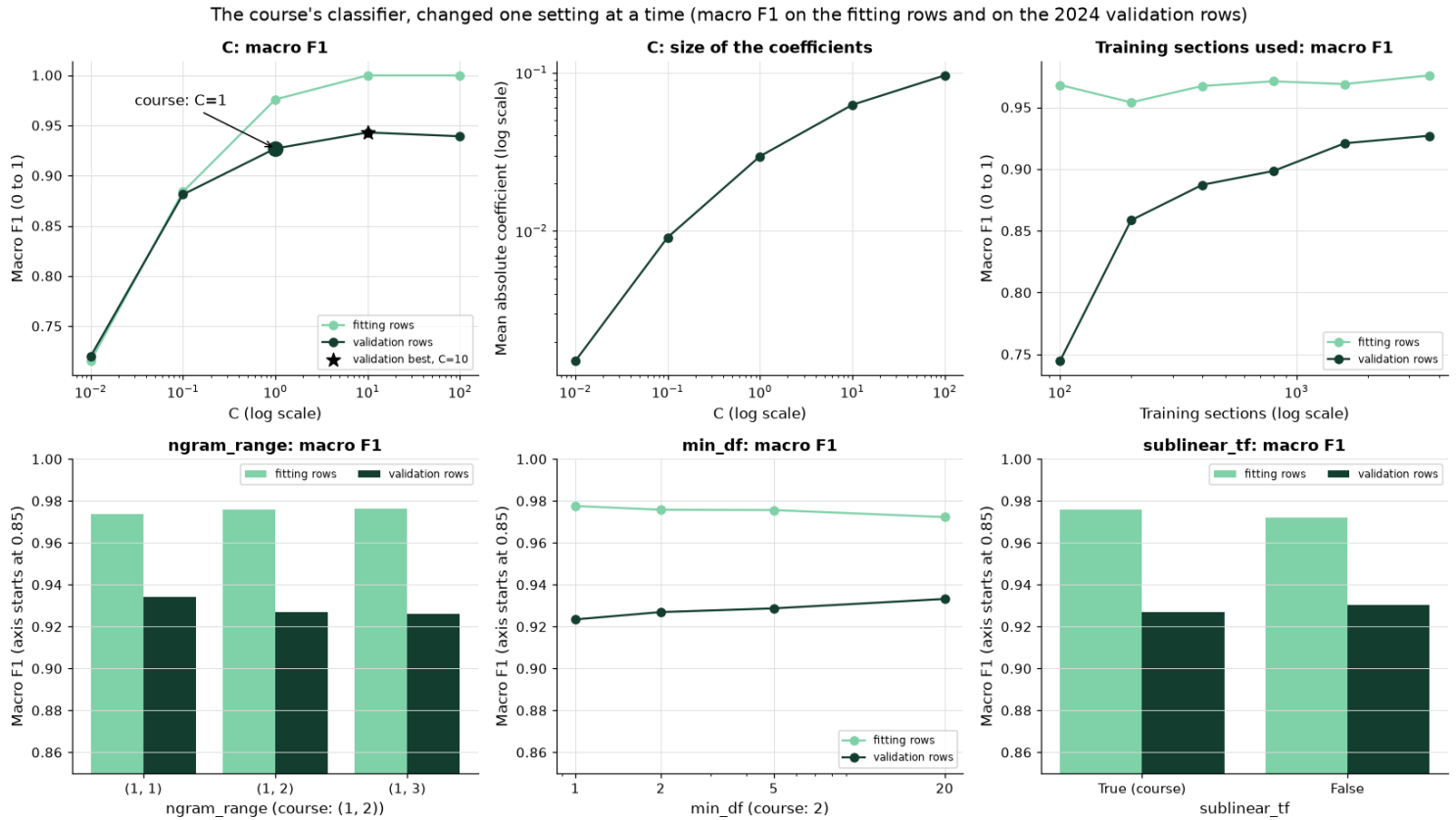
random_state (default None; set once)

The seed for solvers that shuffle. None.

lbfgs does not shuffle; the course passes 42 anyway so no fit depends on luck.

14 settings, read from the library's signature. The course changes two from their defaults and tunes none.

Six pictures of one classifier



0.9997

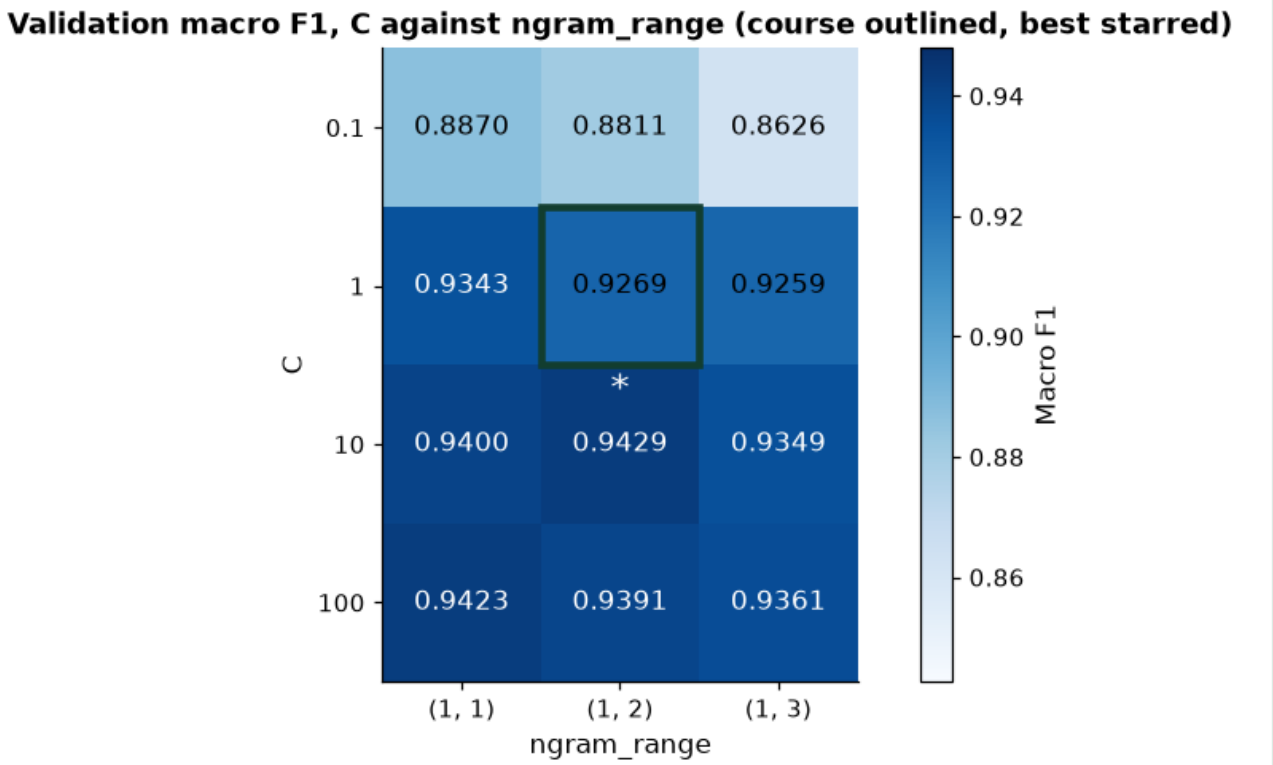
fitting F1 at C=100

0.9429

validation F1 at C=10, best

At C=100 the validation score falls to 0.9391 while the fitting score stays at the ceiling: the overfitting curve.

Two settings together



0.9269

course cell: C=1, pairs

The best cell: C=10, ngram_range=(1, 2), 0.9429. The grid describes this validation year; every setting looked at on the validation rows uses them up a little, so the 2025 rows stay closed until day 10.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIFS(validation!C2:C742,"2.04",validation!D2:D742,"2.04")`	92
`=COUNTIFS(validation!$C$2:$C$742,$A2,validation!$D$2:$D$742,B$1)`	B2:I9, all equal; 741
`=COUNTIFS(validation!$C$2:$C$742,A2,validation!$D$2:$D$742,A2)/COUNTIF(validation!$C$2:$C$742,A2)`	recall, equal
`=2*J2*K2/(J2+K2)`	F1, within 1e-12
`=AVERAGE(L2:L9)`	0.9271
`=SUMPRODUCT(--ISTEXT(validation!C2:D742))`	1,482 text cells

No tokenizer, TF-IDF, or text classifier in Excel. The results table is all COUNTIFS and AVERAGE.

Codes as text, and what COUNTIF does

Written into	Excel holds	ISTEXT
a General cell: 2.10	the number 2.1	FALSE
a Text cell: 2.10	"2.10", LEN 4	TRUE
pandas to_excel: 2.10	"2.10"	TRUE

Three Text cells

`"2.10", "2.1", "2.10"`

Excel side by side (tested in Excel)

`=COUNTIF(Q1:Q3,"2.10")`	3: "2.1" counted
`=COUNTIF(Q1:Q3,"2.1")`	3
`=SUMPRODUCT(--EXACT(Q1:Q3,"2.10"))`	2, as Python's ==

COUNTIF compares number-like text as numbers. EXACT compares the text.

Save the day: the first results.csv

1. Init

the work folder, settings, the guard

2. Code

mlkit and textkit, one commit

3. Log

results.csv, its own commit

4. History

git log -- results.csv

```
.gitignore      |   5 +
mlkit.py        | 568 ++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: a random split: macro F1 0.9339, 40 validation sections share a filing with training; dates 2019-01-03 to 2024-12-30.

Code that runs is not the same as code that is right.

What you can do now

- Read item codes as text, in pandas and Excel
- Split dated filings by date; close the test rows
- Find a heading leak, and fit on the bodies
- Report the rare items' recall on its own
- Build the confusion matrix with COUNTIFS

Week 1, Day 4 Word embeddings